Enum.each

Enum.each/2 is used to iterate over an enumerable. Anything in Elixir that implements the Enumerable Protocol can be enumerated over, but you'll most frequently enumerate lists and maps. Enum.each/2 doesn't return any value from the iteration. Because Elixir is immutable, you cannot modify data that exists outside of the enumeration function.

Let's enumerate a map using Enum.each/2:

```
$ iex
iex> Enum.each(%{a: "value", b: "another"}, fn {key, value} ->
      IO.puts "Key: #{key}; Value: #{value}"
    end)

Key: a; Value: value
Key: b; Value: another
```

Maps are key-value pairs, so the enumeration function accepts each value in a tuple pair. This is an example of how tuples are frequently used for fixed-size data types.

One of the techniques that's common in Ruby apps is to define an empty hash, iterate over some data, build up the hash as you go, and then return the hash. You can't do that in Elixir because data is immutable. The next example demonstrates what happens if you try to. This Elixir code compiles and runs, but the final result is empty:

```
iex> result = %{}

iex> Enum.each([1, 2, 3], fn i ->
  result = Map.put(result, i, true)
end)

iex> result
%{}
```

You'll see a warning that "variable result is unused" when you run this example, and the final result is unchanged. If you want to build up a result, you'll need to use different functions, like Enum.map or Enum.reduce.

Enum.map

Enum.map/2 is one way to transform data. It returns a new list based on the result of the function you give it:

```
$ iex
iex> Enum.map([1, 2, 3], & &1 * 2)
[2, 4, 6]

iex> Enum.map([%{a: 1}, %{a: 2}], & Map.put(&1, :a, "updated"))
[%{a: "updated"}, %{a: "updated"}]
```